Preparing Linux Real-Time Kernel and Tuning Robotics Platform with a Modern ARM64 SoC

Krzysztof Kozlowski
Qualcomm Landing Team, Linaro
krzysztof.kozlowski@linaro.org

Introduction

- Krzysztof Kozlowski
- I work for Linaro in Qualcomm Landing Team / Linaro Developer Services
  - Upstreaming Qualcomm ARM/ARM64 SoCs
  - Maintaining few Linux kernel pieces

- What this talk will not be about
  - What is Real-Time and RTOS
  - PREEMPT_RT patchset

- What this talk will be about
  - Building and configuring a Real-Time Linux kernel
  - What to expect during testing and debugging
  - Basics of tuning the system for Real-Time
  - Evaluation and stress testing on embedded ARM64 robotics platform

Test platform - RB5

- The work I am describing was done on v6.1, but everything applies also to current v6.3
- Qualcomm RB5 Robotics platform
  - ARM64, 8-core SoC QRB5165 (SM8250)
  - 8 GB LPDDR 5 RAM
  - 128 GB UFS storage
  - WiFi, Bluetooth, and so on
  - Compliant with the 96Board

Image source: https://developer.qualcomm.com/qualcomm-robotics-rb5-kit

First steps

- PREEMPT_RT is a patchset aiming to improve Real-Time aspects of the Linux kernel
- Most of it was already merged into mainline, but there are still some tasks to do
  - One can get the PREEMPT_RT from Git repo or as patchset for git-am
    - Remember to get Sebastian Andrzej Siewior's key from kernel.org keyring
    - pgpkeys/keys/7B96E8162A8CF5D1.asc
  - See https://wiki.linuxfoundation.org/realtime/ for details

Kernel build configuration

- CONFIG_PREEMPT_RT=y
  - Fully Preemptible Kernel (Real-Time)
  - $ cat /sys/kernel/realtime
- CONFIG_NO_HZ_FULL=y
  - Which will behave as NO_HZ_IDLE by default
- CONFIG_HZ_1000=y
- CONFIG_CPUSETS=y
  - For isolating CPUs for Real-Time workloads
- CONFIG_BLK_CGROUP_IOLATENCY=y

Most likely you will also want for evaluation and debugging latency issues:

- CONFIG_LATENCYTOP=y
- CONFIG_SCHED_TRACER=y
- CONFIG_TIMERLAT_TRACER=y
- CONFIG_HWLAT_TRACER=y

What can go wrong

- PREEMPT_RT will likely exercise a bit different driver paths in regard of concurrency
- Thus new race conditions are possible due to:
  - Missing synchronization
  - Different code-flow, e.g. order of driver callbacks between devices
- Build a test kernel with:
  - CONFIG_KASAN=y
  - CONFIG_DEBUG_SHIRQ=y
  - CONFIG_SOFTLOCKUP_DETECTOR=y
  - CONFIG_DETECT_HUNG_TASK=y
  - CONFIG_WQ_WATCHDOG=y
  - CONFIG_DEBUG_PREEMPT=y
  - CONFIG_DEBUG_IRQFLAGS=y
- Look for usual splats like:
  - Unable to handle kernel paging request at virtual address
  - BUG: KASAN: use-after-free in tty_driver_flush_buffer

What can go wrong

- Typical problem is a direct result of PREEMPT_RT: spinlock and few more locks are now sleeping primitives
- For example the spinlock should not be used within atomic sections:
  - Disabled interrupts
  - Disabled preemption
  - Instead one could use raw_spinlock
  - It is even trickier with local_lock(), but that's not a typical case, so out of scope
- Build a test kernel with LOCKDEP:
  - CONFIG_PROVE_LOCKING=y
    - Lock debugging: prove locking correctness
  - CONFIG_PROVE_RAW_LOCK_NESTING=y
    - Enable raw_spinlock - spinlock nesting checks
  - CONFIG_DEBUG_ATOMIC_SLEEP=y
    - Sleep inside atomic section checking

What can go wrong - disabled IRQs

- Look for:
  - BUG: sleeping function called from invalid context at kernel/locking/spinlock_rt.c:46
    in_atomic(): 0, irqs_disabled(): 128, non_block: 0, pid: 298, name: systemd-udevd
    preempt_count: 0, expected: 0
- Non-PREEMPT_RT correct but PREEMPT_RT incorrect:

```
local_irq_disable();
...
  spin_lock_irqsave(&l, flags);
  ...
  spin_unlock_irqrestore(&l, flags);
...
local_irq_enable();
```

Both correct (example approach):

```
local_irq_disable();
...
  raw_spin_lock_irqsave(&l, flags);
  ...
  raw_spin_unlock_irqrestore(&l, flags);
...
local_irq_enable();
```

What can go wrong - disabled preemption

- Look for:
  - BUG: sleeping function called from invalid context at kernel/locking/spinlock_rt.c:46
    in_atomic(): 1, irqs_disabled(): 0, non_block: 0, pid: 291, name: systemd-udevd
    preempt_count: 1, expected: 0
- Non-PREEMPT_RT correct but PREEMPT_RT incorrect:

```
preempt_disable();
...
  spin_lock_irqsave(&l, flags);
  ...
  spin_ublock_irqrestore(&l, flags);
...
preempt_enable();
```

Both correct:

```
preempt_disable();
...
  raw_spin_lock_irqsave(&l, flags);
  ...
  raw_spin_unlock_irqrestore(&l, flags);
...
preempt_enable();
```

What can go wrong - memory allocation

- Memory allocator is now fully preemptible, also for GFP_ATOMIC
- Look for:
  - BUG: sleeping function called from invalid context
- Non-PREEMPT_RT correct but PREEMPT_RT incorrect:

```
raw_spin_lock(&l);
p = kmalloc(sizeof(*p), GFP_ATOMIC);
...
raw_spin_unlock(&l);
```

Both correct:

```
spin_lock(&l);
p = kmalloc(sizeof(*p), GFP_ATOMIC);
...
spin_unlock(&l);
```

- ... or move the allocation out of critical section

System Tuning

Tuning the system

- Kernel with PREEMPT_RT is not enough
- Several regular kernel activities (housekeeping tasks) can interrupt Real-Time application adding unexpected latencies
  - RCU callbacks
  - Periodic timer ticks
  - Interrupts
  - Workqueues
- Also Real-Time application should not fight with other processes for CPU time
- Usually some CPUs are assigned to housekeeping tasks and some to Real-Time
  - E.g. CPU 0-1 for housekeeping, rest (CPU 2-7) for RT

Tuning the system - command line

- Offload RCU callbacks from RT CPUs:
  - rcu_nocbs=2-7 rcu_nocb_poll
- Default IRQ affinity to housekeeping CPUs:
  - irqaffinity=0-1
- Mitigate for xtime_lock contention:
  - skew_tick=1
- Disable lockup detectors:
  - nosoftlockup nowatchdog
- For specific workloads (one thread per CPU core) disable tick on RT CPUs:
  - nohz_full=2-7
  - Long latency penalty during context switches, thus it must match specific workload

Tuning the system - runtime

- Keep IRQs on housekeeping CPUs:
  - systemctl disable irqbalance
  - Or use IRQBALANCE_BANNED_CPUS so they will be balanced between housekeeping CPUs (e.g. to still distribute busy UFS and USB/Ethernet interrupts among two CPUs)
- Move workqueues to housekeeping CPUs:
  - echo 03 > /sys/devices/virtual/workqueue/blkcg_punt_bio/cpumask
    echo 03 > /sys/devices/virtual/workqueue/scsi_tmf_0/cpumask
    echo 03 > /sys/devices/virtual/workqueue/writeback/cpumask
  - And possibly other...
- Disable CPU frequency scaling
  - cpupower frequency-set -g performance
- Disable deeper CPU idle states
  - cpupower idle-set -d 1
- Allowing RT application up to 100% of CPU time (optional)
  - /proc/sys/kernel/sched_rt_runtime_us
  - Other tasks can starve

Tuning the system - cpusets

- Older kernels used “isolcpus” command line argument
- Since some time, cgroups/cpusets should be used
  - For instructions see: https://docs.kernel.org/admin-guide/cgroup-v2.html#cpuset

```
cd /sys/fs/cgroup/
echo "+cpuset" >> /sys/fs/cgroup/cgroup.subtree_control

# Create housekeeping cpuset for CPU 0-1:
mkdir /sys/fs/cgroup/bulk
echo "+cpuset" >> bulk/cgroup.subtree_control
echo 0-1 >> bulk/cpuset.cpus
ps -eLo lwp | while read thread; do echo $thread > bulk/cgroup.procs ; done
```

Tuning the system - cpusets (continued)

- Now the Real-Time group:

```
mkdir /sys/fs/cgroup/rt
# Consider "isolated" partition, but then tasks won't be balanced
# echo isolated > rt/cpuset.cpus.partition
echo root > rt/cpuset.cpus.partition
echo "+cpuset" >> rt/cgroup.subtree_control
echo "2-7" >> rt/cpuset.cpus

# Test if group has correct (not invalid) configuration
cat rt/cpuset.cpus.partition
-> expected: root

# Run your app with:
cgexec -g cpuset:rt ..........
```

System Evaluation

Evaluation of the system

- Real-Time use case requires application to respond to event within some deadline
- Latency: time between event and actual response
- For your workload, real or simulated, you might need to know what is the maximum experienced latency
- The typical tools for this are cyclictest and stress-ng
  - cyclictest - application measuring latencies in real-time systems caused by the hardware, the firmware, and the operating system.
  - stress-ng - stressor of various parts of system, includes also cyclic functionality
  - rtla timerlat - cyclictest on steroids, using kernel tracers
- E.g. make your RT CPUs busy at 60% and measure latencies with cyclictest

```
cgexec -g cpuset:rt stress-ng --cpu 6 --cpu-load 60
cgexec -g cpuset:rt cyclictest -m --affinity 7 --threads 1 -p 95 -h 150 \
    --mainaffinity=2 --policy fifo
```

Evaluation of the system - cyclictest output

```
policy: fifo: loadavg: 0.16 2.84 3.25 1/546 4520
T: 0 ( 4515) P:95 I:1000 C:  37859 Min:      6 Act:    6 Avg:    6 Max:      11
T: 1 ( 4516) P:95 I:1000 C:  37859 Min:      6 Act:    6 Avg:    6 Max:       9
T: 2 ( 4517) P:95 I:1000 C:  37858 Min:      2 Act:    2 Avg:    2 Max:       4
T: 3 ( 4518) P:95 I:1000 C:  37858 Min:      2 Act:    2 Avg:    2 Max:       4
T: 4 ( 4519) P:95 I:1000 C:  37858 Min:      2 Act:    2 Avg:    2 Max:       5
T: 5 ( 4520) P:95 I:1000 C:  37858 Min:      2 Act:    2 Avg:    2 Max:       4
# Histogram
000000 000000    000000     000000     000000     000000     000000
000001 000000    000000     000000     000000     000000     000000
000002 000000    000000     036446     036036     033388     037843
000003 000000    000000     001413     001823     004466     000016
...
# Min Latencies: 00006 00006 00002 00002 00002 00002
# Avg Latencies: 00006 00006 00002 00002 00002 00002
# Max Latencies: 00011 00009 00004 00004 00005 00004
```

Evaluation of the system

- Qualcomm RB5 Robotics platform example latencies
  - ARM64, 8-core SoC QRB5165 (SM8250)
  - 6.1.7-rt5, Qualcomm Landing Team kernel
    - v6.1 kernel with PREEMPT_RT patches
    - With some hardware enablement patches being upstreamed
    - With Real-Time fixes developed during entire process (already upstreamed or in process)
    - Should be without differences against current mainline (-PREEMPT_RT)
- Compared two kernels
  - RT: v6.1.7-rt5 kernel, PREEMPT_RT
  - Vanilla: same kernel, built with PREEMPT (“Preemptible Kernel (Low-Latency Desktop)”)
- Both systems tuned (e.g. cmdline, isolated CPUs)

Evaluation of the system - Qualcomm RB5

cyclictest on CPU7	Min latency [us]	Average lat. [us]	Max latency [us]
Vanilla, idle	1	2	4
RT, idle	1	1	5
Vanilla, 60% load	2	3	9
RT, 60% load	2	2	10
Vanilla, 100% load	2	2.5	7
RT, 100% load	2	2	6

Evaluation of the system - Qualcomm RB5

- Heterogeneous systems will have different latency results on different cores

cyclictest on CPU2-7	Min latency [us]	Average lat. [us]	Max latency [us]
CPU	2, 3, 4, 5, 6, 7	2, 3, 4, 5, 6, 7	2, 3, 4, 5, 6, 7
RT, idle	5, 6, 2, 1, 2, 1	7, 6, 2, 2, 2, 1	19, 15, 5, 4, 6, 4
RT, 100% load	5, 5, 2, 2, 2, 2	6, 6, 3, 3, 3, 2	19, 23, 7, 7, 7, 6

Latency spikes - hwlatdetect

- What if the average latency is low, but the maximum is high?
- Check latencies introduced by hardware or firmware with **hwlatdetect**
  - On RT/isolated CPUs

```
hwlatdetect  --duration=600s --cpu-list=2-7 --threshold=5
   parameters:
        CPU list:          2-7
        Latency threshold: 5us
        Sample window:     1000000us
        Sample width:      500000us
     Non-sampling period:  500000us
        Output File:       None
Max Latency: Below threshold
Samples recorded: 0
Samples exceeding threshold: 0
```

Latency spikes - tracing

- Cyclictest can help trace the cause of the latency
  - First set up your tracing
  - Then cyclictest with “-b XX --tracemark” argument

```
cd /sys/kernel/tracing/
echo function > current_tracer
echo 1 > tracing_on
cgexec -g cpuset:rt cyclictest -m --affinity 7 --threads 1 -p 95 -h 150 \
    --mainaffinity=2 --policy fifo -b 25 --tracemark

less trace # look for tracing_mark_write
```

Latency spikes - rtla osnoise

- Look for OS noise with rtla
  - apt-get install rtla
  - Or build it from linux/tools/tracing/rtla
- rtla osnoise gives answers about noise caused by the system
- How much of time is taken from RT application, e.g. by IRQs or preemption?
- Look for noise on isolated CPUs
- Refer to RTLA: Real-time Linux Analysis Toolset - Daniel Bristot De Oliveira, Red Hat for tutorial/howto

```
$ rtla osnoise top --stop 10 --threshold 5 --cpus 2-7 --trace
CPU Period       Runtime        Noise  % CPU Aval   Max Noise   Max Single
  2 #4           4000000         6664    99.83340        2075           67
  3 #4           4000000          472    99.98820         263           19
  4 #4           4000000            0   100.00000           0            0
  5 #4           4000000         6542    99.83645        2170          147
  6 #4           4000000          155    99.99612          54           54
  7 #4           4000000           15    99.99962          15           15
```

Latency spikes - rtla timerlat

- rtla timerlat is a cyclictest on steroids
  - Refer to RTLA: Real-time Linux Analysis Toolset - Daniel Bristot De Oliveira, Red Hat

```
rtla timerlat top --cpus 2-7 --auto 25
## CPU 2 hit stop tracing, analyzing it ##
  IRQ handler delay:                                1.23 us (4.85 %)
  IRQ latency:                                      5.24 us
  Timerlat IRQ duration:                           10.47 us (41.31 %)
  Blocking thread:                                  6.62 us (26.10 %)
                     swapper/2:0                         6.62 us
    Blocking thread stack trace
           -> timerlat_irq
           -> __hrtimer_run_queues
           -> hrtimer_interrupt
           -> arch_timer_handler_virt
           -> handle_percpu_devid_irq
```

Resources and references

- cylictest
- Optimizing RHEL 8 for Real Time for low latency operation
- RTLA: Real-time Linux Analysis Toolset - Daniel Bristot De Oliveira, Red Hat

Linaro Connect
LONDON 2023

Thank you

Presentations will be available post event at
Resource Hub